

Overview

Part 08 —  Retrieval-Augmented Generation · 08-rag/README.md



Module 08 — Retrieval-Augmented Generation

Where you are: Stop 8 of 16. **Time:** ~22 hours · **Prereq:** Module 04 — LLM [04-llm] (context windows, prompting) and Module 03 — NLP [03-nlp] (embeddings) — you need both before this makes sense.

An LLM's knowledge is frozen at training time and it cannot see your private data. RAG is the practical fix: retrieve relevant text at query time and hand it to the model as context, instead of (or alongside) fine-tuning or stuffing everything into an ever-longer context window. It is the single most-deployed LLM architecture in production for one reason — it gives you fresh knowledge, source citations, and access control over private data, all without retraining anything. It is also where most "the AI gave a wrong answer" tickets actually originate, and almost never in the LLM: they originate in the chunker, the index, or the retriever, three components most people never think to debug.

Learning objectives

1. Explain precisely why RAG exists — and defend the choice between RAG, long-context, and fine-tuning for a given problem on cost and freshness grounds, not vibes.
2. Chunk a real document with the strategy matched to its structure (fixed-size, recursive, sentence-window, structure-aware) and explain which one prevents a fact from being split across a boundary.
3. Reason quantitatively about vector search: exact kNN vs ANN (HNSW, IVF), and the recall/latency/memory triangle you trade against as a corpus grows.
4. Combine sparse (BM25) and dense retrieval with Reciprocal Rank Fusion, then sharpen the fused list with a cross-encoder rerank — and say why hybrid rarely loses to either method alone.
5. Pick the right advanced retrieval pattern (query rewriting, HyDE, multi-query, parent-document, GraphRAG) for a stated failure mode, not by default.
6. Evaluate a RAG pipeline with real numbers: Recall@k, MRR, and NDCG@k for retrieval; a faithfulness/groundedness check for generation — and know why you need both halves.
7. Diagnose the five recurring production RAG failure modes (lost-in-the-middle, stale indexes, chunk-boundary splits, retrieval-generation mismatch, silent index drift) and propose a concrete fix for each.

Suggested path

#	Do this	File	Time
1	Why RAG, chunking, embeddings, vector search mechanics	notes/01-core-concepts.md	4h
2	Hybrid search, rerank, advanced patterns, GraphRAG, eval, failure modes	notes/02-advanced-patterns-and-evaluation.md	5h
3	Run the chunking strategies comparison	code/chunking_strategies.py	2h
4	Run hybrid search (BM25 + dense + RRF + rerank)	code/hybrid_search_and_rerank.py	2h
5	Run retrieval + faithfulness eval metrics	code/rag_eval_metrics.py	2h
6	Slides	slides/	1h
7	Lab	lab/EXERCISES.md	6h
8	Papers	papers/PAPERS.md	5h

The 14 terms you must own

chunking · embedding · ANN index (HNSW / IVF) · cosine similarity · BM25 · hybrid search · Reciprocal Rank Fusion (RRF) · cross-encoder rerank · HyDE · query rewriting · parent-document retrieval · GraphRAG · faithfulness / groundedness · lost-in-the-middle

Exit check

You can hand a colleague a small end-to-end RAG eval: a chunker with a stated boundary strategy, a hybrid (BM25 + dense) retriever fused with RRF and sharpened with a rerank pass, a labeled query set scored with Recall@k / MRR / NDCG@k, a faithfulness score for the generated answers, and one paragraph naming the single biggest failure mode you'd fix next and why.

Where this connects: embeddings and cosine similarity come from Module 03 — NLP [03-nlp]; context limits and lost-in-the-middle degradation are a Module 04 — LLM [04-llm] phenomenon that RAG must design around; letting an agent decide *when* and *what* to retrieve, iteratively, is Module 06 — AI Agents [06-ai-agents]' territory (agentic/iterative RAG); and the RAGAS-style metrics in this module get their full production treatment — statistical rigor, LLM-as-judge, regression testing — in Module 15 — AI Evals [15-ai-evals].